

COMPOSE -- PROD STAGE 2

Instrucciones

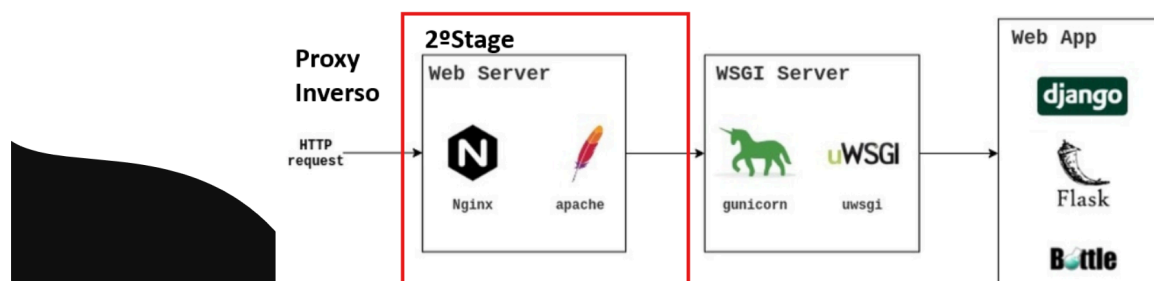
Amplía la entrega de [Prod Stage 1](#) del blog que Laura hizo con Lidia en Backend de segundo de DAW. En esta tercera y última etapa conservas Gunicorn y Django e incorporas **Nginx como proxy inverso y servidor de archivos estáticos y multimedia**.

Objetivo de esta etapa

Completa la parte recuadrada en rojo en el esquema: Nginx recibe las peticiones del navegador, sirve directamente los archivos de `/static/` y `/media/`, y envía las peticiones de la aplicación al servicio interno web:8000.

Compose – prod deploy 2ºstage

- +Problemas → gunicorn NO es capaz de servir ficheros estáticos.
- CSS, Tailwind: No disponible.
- Deberemos servirlos con un proxy inverso:



Nginx se incorpora al trabajo de la etapa anterior.

1. Añadir el servicio Nginx

Amplía Compose con un servicio Nginx que escuche en el puerto 80 y comparta la red con el servicio web de Gunicorn. Publica el acceso del navegador a través de Nginx y conserva el puerto 8000 para la comunicación interna con la aplicación.

Crea la configuración local `nginx/conf.d/default.conf` y móntala en el directorio `/etc/nginx/conf.d/` del contenedor. Configura estos destinos:

- `/static/`: sirve directamente los archivos de `/usr/src/app/staticfiles/` mediante alias.
- `/media/`: sirve directamente los archivos de `/usr/src/app/media/` mediante alias.

- /: utiliza proxy_pass http://web:8000 para las peticiones de Django. web debe coincidir con el nombre de su servicio en Compose.

Incluye las cabeceras de la captura: Host con \$host, X-Real-IP con \$remote_addr, X-Forwarded-For con \$proxy_add_x_forwarded_for y X-Forwarded-Proto con \$scheme.

Compose – prod deploy 2ºstage

- Configuración de nginx en local: ./nginx/conf.d/default.conf

/static/ → No proxy

/media/ → No proxy

location / → proxy a la dirección interna
http://web:8000

```
server {
    listen 80;
    server_name _;

    location /static/ {
        alias /usr/src/app/staticfiles;
    }

    location /media/ {
        alias /usr/src/app/media;
    }

    location / {
        proxy_pass http://web:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Los estáticos y los archivos subidos se sirven directamente; las demás peticiones pasan a Unicorn.

2. Reunir los archivos estáticos

En el proyecto original, los recursos están repartidos entre static/, los estáticos de las aplicaciones y theme/static/css/dist/. Prepara las dependencias de theme/static_src/ y ejecuta su compilación de producción de Tailwind, definida en el script npm run build, antes de recopilar los estáticos.

Compose – prod deploy 2ºstage

- Django y Tailwind → Dependencias estáticas repartidas entre múltiples directorios

Identifica los directorios de origen de los estáticos del blog.

Añade en personalblog/settings.py el directorio de destino de la recopilación:

```
STATIC_ROOT = BASE_DIR / "staticfiles"
```

Ejecuta python manage.py collectstatic --noinput en el contenedor de Django con el volumen de estáticos montado. Comprueba que reúne el CSS compilado, JavaScript, imágenes estáticas y los recursos de las aplicaciones en staticfiles/. La carpeta de destino es distinta de las carpetas de origen; media/ se comparte por separado.

Compose – prod deploy 2ºstage

● `settings.py` → `STATIC_ROOT = BASE_DIR / "staticfiles"`

Define la carpeta donde Django volcará todos los archivos estáticos cuando ejecutes `python manage.py collectstatic`

Configura el destino de `collectstatic` y verifica su contenido.

3. Compartir los archivos entre los contenedores

Declara un volumen con nombre `staticfiles` a nivel global en Compose y móntalo en ambos servicios. Siguiendo las rutas de las capturas:

- Django: `staticfiles:/app/staticfiles`, si la raíz del proyecto definida en el Dockerfile es `/app`. Adapta esta ruta si tu directorio de trabajo es otro: debe coincidir con `STATIC_ROOT`.
- Nginx: `staticfiles:/usr/src/app/staticfiles:ro`, para leer los archivos que ha recopilado Django.
- Comparte también el mismo contenido de `media/`: por ejemplo, `./media:/app/media` en Django y `./media:/usr/src/app/media:ro` en Nginx. Conserva las imágenes incluidas en el proyecto.

Las rutas de montaje pueden ser distintas en cada contenedor, pero deben apuntar al mismo volumen o a la misma carpeta compartida. Los destinos de Nginx deben coincidir con sus directivas `alias`.

Compose – prod deploy 2ºstage

- Volúmenes: Volumen global compartido entre 2 contenedores y montado en distintas rutas.

/app(La raíz definida en el dockerfile)/staticfiles → por STATIC_ROOT

staticfiles:/usr/src/app/staticfiles → Nginx lee los archivos que Django generó con

collectstatic en el otro contenedor.

```
volumes:
  - staticfiles:/app/staticfiles
expose:
volumes:
  - ./nginx/conf.d:/etc/nginx/conf.d
  - staticfiles:/usr/src/app/staticfiles
  - ./media:/usr/src/app/media
from:
volumes:
  staticfiles:
```

Volúmenes compartidos y correspondencia entre las rutas de Django y Nginx.

Comprobaciones y entrega

Comprueba el blog accediendo por Nginx: las páginas de Django deben responder, los estilos de Tailwind y las imágenes deben cargarse, y las peticiones de /static/ y /media/ deben devolver los archivos esperados. Mantén la configuración de producción y la retirada de django-browser-reload de la etapa anterior.

Adjunta el enlace al mismo repositorio actualizado y amplía el README.md con los cambios respecto a Prod Stage 1: servicios, configuración de Nginx, compilación de Tailwind, STATIC_ROOT, ejecución de collectstatic y montajes. Incluye capturas del blog completo, de ambos contenedores, de los archivos recopilados y de las peticiones de estáticos y multimedia funcionando.

Materiales de la tarea

Los archivos adjuntos se descargan por separado desde la página de esta tarea.



[laura-personal_blog-main.zip](#)

Proyecto original de referencia · continúa sobre tu entrega de Prod Stage 1 · ZIP, 2,6 MB

